1. BACKEND ARCHITECTURE

SAMS (Student Attendance Management System) - PHP Backend

Table of Contents

Overview

SAMS Backend is a RESTful PHP application built with modern architecture principles: - **Framework**: Pure PHP 7.4+ with PDO for database abstraction - **Architecture**: MVC (Model-View-Controller) with Service Layer - **Database**: MySQL/MariaDB (multi-database support including PostgreSQL) - **Authentication**: Session-based with JWT support - **API Format**: JSON REST - **Deployment**: Heroku, Azure, Docker support

Core Features: - User authentication (Admin, Teacher, Student roles) - Attendance marking with GPS + Face verification - Schedule management - Real-time notifications (FCM) - Reports and analytics - System settings management

---

Project Structure

```
sams-backend/
├── config/
│   ├── database.php           # PDO database connection & configuration
│   ├── constants.php          # Global constants, thresholds, timeouts
│   ├── firebase.php           # Firebase/FCM configuration
│   ├── fcm-helper.php         # FCM notification helper functions
│   └── schema.sql             # Database schema (MySQL)
│
├── includes/
│   ├── controllers/           # Business logic layer
│   │   ├── AuthController.php
│   │   └── UserController.php
│   │
│   ├── models/                # Data access layer (ORM)
│   │   ├── User.php
│   │   ├── Student.php
│   │   ├── Teacher.php
│   │   ├── Attendance.php
│   │   ├── Schedule.php
│   │   ├── Subject.php
│   │   ├── Department.php
│   │   ├── Notification.php
│   │   └── ...
│   │
│   ├── helpers/               # Utility functions
```

```
│   │   ├── Response.php       # Standardized JSON responses
│   │   ├── Validator.php      # Input validation
│   │   ├── DateHelper.php     # Date/time utilities
│   │   ├── GpsHelper.php      # GPS distance calculation
│   │   ├── EncryptionHelper.php
│   │   └── ...
│   │
│   └── middleware/            # Cross-cutting concerns
│       ├── Auth.php           # Authentication & authorization
│       ├── CORS.php           # CORS headers handling
│       └── RateLimit.php      # Rate limiting
│
├── api/                       # Public API endpoints (routing)
│   ├── admin/                 # Admin APIs
│   │   ├── users.php
│   │   ├── departments.php
│   │   ├── subjects.php
│   │   ├── teacher-assignments.php
│   │   ├── schedules.php
│   │   ├── settings.php
│   │   ├── reports.php
│   │   └── ...
│   │
│   ├── teacher/               # Teacher APIs
│   │   ├── dashboard.php
│   │   ├── profile.php
│   │   ├── schedule.php
│   │   ├── start-class.php
│   │   ├── end-class.php
│   │   ├── class-attendance.php
│   │   ├── manual-attendance.php
│   │   └── ...
│   │
│   ├── student/               # Student APIs
│   │   ├── dashboard.php
│   │   ├── profile.php
│   │   ├── schedule.php
│   │   ├── register-face.php
│   │   ├── verify-face.php
│   │   ├── mark-attendance.php
│   │   ├── attendance-history.php
│   │   └── ...
│   │
│   ├── fcm/                   # Firebase Cloud Messaging
│   │   └── register-token.php
│   │
│   ├── public/                # Public endpoints (no auth required)
│   │   └── index.php
│   │
│   ├── health-check.php       # System health status
│   └── test-db.php            # Database connection test
│
├── public/                    # Web root (served by web server)
│   ├── index.php              # Main entry point
│   ├── login.php              # Legacy login page
│   └── assets/                # Static files (CSS, JS, images)
│
├── logs/                      # Application logs
├── uploads/                   # User uploads (profile photos, etc.)
├── vendor/                    # Composer dependencies
│
├── composer.json              # PHP dependencies
├── .env                       # Environment variables
├── Dockerfile                 # Docker configuration
├── docker-compose.yml         # Docker Compose setup
```

```
├── Procfile                    # Heroku deployment config
├── deploy-azure.sh             # Azure deployment script
└── README.md                   # Project documentation
```

Architecture Layers

1. Presentation Layer (API Endpoints)

Location: `/api/**/*.php`

Handles HTTP requests and returns JSON responses: - Validates HTTP method (GET, POST, PUT, DELETE) - Calls middleware (CORS, Auth) - Routes to controller methods - Returns standardized JSON responses

Example: `/api/student/dashboard.php`

```
header('Content-Type: application/json');
require_once __DIR__ . '/../config/database.php';
require_once __DIR__ . '/../includes/middleware/Auth.php';

// 1. CORS handling
CORS::handle();

// 2. Authentication
$user = Auth::user();
if (!$user) {
    Response::unauthorized('Login required');
}

// 3. Route to controller
$controller = new StudentController($db);
$controller->getDashboard($user['id']);
```

2. Business Logic Layer (Controllers)

Location: `/includes/controllers/*.php`

Contains core business logic and orchestrates models: - Validates input data - Calls model methods - Handles business rules - Performs error handling - Prepares response data

Responsibilities: - Authentication & authorization checks - Data validation & transformation - Calling appropriate models - Error handling and logging - Response formatting

3. Data Access Layer (Models)

Location: `/includes/models/*.php`

Provides database abstraction and CRUD operations:

Base Model Pattern:

```
class Model {
    protected $db;
    protected $table;

    public function __construct($db) {
        $this->db = $db;
    }
```

```
    public function getById($id) { ... }
    public function getAll() { ... }
    public function create($data) { ... }
    public function update($id, $data) { ... }
    public function delete($id) { ... }
}
```

Key Models: - `User.php` - User authentication & profile - `Student.php` - Student data & face registration - `Teacher.php` - Teacher data & assignments - `Attendance.php` - Attendance records - `Schedule.php` - Class schedules - `Notification.php` - System notifications - `Department.php`, `Subject.php` - Master data

4. Helper/Utility Layer

Location: `/includes/helpers/*.php`

Provides reusable functions:

- **Response.php**: Standardized JSON responses

  ```
  Response::success($data, 'Message');
  Response::error('Error message', 400);
  Response::validationError($errors);
  ```

- **Validator.php**: Input validation

  ```
  $validator->required('email', $data['email'], 'Email');
  $validator->email('email', $data['email']);
  $validator->numeric('id', $data['id']);
  ```

- **GpsHelper.php**: Distance calculation between coordinates
- **EncryptionHelper.php**: Face data encryption (AES-256)
- **DateHelper.php**: Date/time conversions
- **TokenHelper.php**: JWT token generation

5. Middleware Layer

Location: `/includes/middleware/*.php`

Cross-cutting concerns:

- **Auth.php**: Session management & role verification

  ```
  Auth::user()              // Get current user
  Auth::hasRole('teacher')  // Check role
  Auth::logout()            // Destroy session
  ```

- **CORS.php**: Cross-Origin Resource Sharing

  ```
  CORS::handle()  // Set CORS headers
  ```

- **RateLimit.php**: Request rate limiting

Core Components

Authentication System

Session-Based Authentication: 1. User login with email/password 2. Password verified using `password_verify()` 3. Session created with unique session_id 4. Session stored in `sessions` table with IP & user agent 5. Subsequent requests validated using session

Workflow:

```
// Login
$user = User::verifyPassword($email, $password);
$sessionId = bin2hex(random_bytes(32));
$_SESSION['user_id'] = $user['id'];
$_SESSION['role'] = $user['role'];

// Verify Session (on each API call)
$user = Auth::user();
if (!$user) { Response::unauthorized('Login required'); }

// Authorization Check
if ($user['role'] !== 'teacher') {
    Response::error('Access restricted to teachers', 403);
}
```

Roles & Permissions: - **Admin**: Full system access, manage users, reports, settings - **Teacher**: Manage classes, mark attendance, view reports - **Student**: View schedule, mark attendance, view own attendance

Attendance System

Attendance Verification: Uses dual verification for security:

1. **GPS Verification**
   - Calculate distance between student and teacher location
   - Threshold: 50 meters (configurable)
   - Status: ‘gps_failed’ if distance > threshold
2. **Face Verification**
   - Compare face embedding from Android app
   - Confidence score threshold: 75% (configurable)
   - Status: ‘face_failed’ if confidence < threshold
3. **Final Status**:
   - ‘success’: Both GPS & Face passed
   - ‘gps_failed’: GPS out of range
   - ‘face_failed’: Face confidence too low
   - ‘both_failed’: Both checks failed

Attendance Marking Flow:

```
1. Teacher starts class → creates session in teacher_locations
2. Student marks attendance → GPS + Face verified
3. Database stores: location, face_score, verification_status
4. Teacher ends class → auto-marks absent for non-present students
5. Reports generated from attendance records
```

Notification System

FCM (Firebase Cloud Messaging) Integration: - Android app registers FCM token - Backend sends push notifications for: - Attendance alerts - Low attendance warnings - Schedule changes - System announcements - Face re-registration required

Notification Types:

```
- 'attendance_alert': Session started, mark attendance reminder
- 'low_attendance': Student below threshold
- 'schedule_change': Class schedule updated
- 'face_reregister': Face registration expired
- 'system': System announcements
```

Design Patterns

1. MVC Pattern

- **Model**: Database layer (Student, Teacher, Attendance models)
- **View**: JSON responses (no templates, API-only)
- **Controller**: Business logic (StudentController, TeacherController)

2. Repository Pattern

Models act as repositories:

```
class Attendance {
    public function getByStudent($studentId, $limit = 50) { ... }
    public function createRecord($data) { ... }
    public function updateStatus($attendanceId, $status) { ... }
}
```

3. Singleton Pattern

Database connection:

```
class Database {
    private static $instance;

    public static function getInstance() {
        if (!self::$instance) {
            self::$instance = new self();
        }
        return self::$instance;
    }
}
```

4. Factory Pattern

Response handling:

```
class Response {
    public static function success($data, $message = null) { ... }
    public static function error($message, $code = 400) { ... }
    public static function validationError($errors) { ... }
}
```

5. Strategy Pattern

Database abstraction (MySQL vs PostgreSQL):

```
public function getConnection() {
    if ($this->db_type === 'postgres') {
        $dsn = "pgsql:host={$host};dbname={$db_name}";
    } else {
        $dsn = "mysql:host={$host};dbname={$db_name}";
    }
    return new PDO($dsn, $user, $pass);
}
```

Configuration & Deployment

Environment Variables

.env file (local development):

```
MYSQL_HOST=localhost
MYSQL_DATABASE=sams_db
MYSQL_USER=root
MYSQL_PASSWORD=

FIREBASE_API_KEY=your_firebase_key
FIREBASE_MESSAGE_SENDER_ID=your_sender_id

SESSION_LIFETIME=3600
SESSION_DOMAIN=.example.com
CORS_ORIGIN=http://localhost:3000
```

Production Deployment

Constants (`config/constants.php`):

```
// Session
define('SESSION_LIFETIME', 3600);          // 1 hour
define('SESSION_DOMAIN', '.sams.edu');

// Attendance
define('GPS_RADIUS_METERS', 50);           // GPS threshold
define('FACE_CONFIDENCE_THRESHOLD', 75);   // Face score threshold
define('LATE_THRESHOLD_MINUTES', 15);      // Late threshold

// System
define('PAGINATION_LIMIT', 50);
define('MAX_UPLOAD_SIZE', 5242880);        // 5MB
```

Error Handling

PDO Exceptions:

```
try {
    $stmt = $db->prepare($query);
    $stmt->execute($params);
} catch (PDOException $e) {
    error_log('DB Error: ' . $e->getMessage());
    Response::error('Database error', 500);
}
```

Validation Errors:

```
$validator->required('email', $data['email'] ?? '', 'Email');
if ($validator->hasErrors()) {
    Response::validationError($validator->getErrors());
    // Returns: { "success": false, "errors": { ... } }
}
```

Performance Optimization

Database Indexing: - Composite indexes on frequently queried columns - Foreign key indexes for joins - Unique indexes on primary identifiers

Query Optimization: - Prepared statements (prevent SQL injection) - Selective column selection (avoid SELECT *) - Pagination for large datasets - Join optimization for related data

Caching Strategies: - Session data caching - Settings cached in-memory - API response caching for static data

---

Summary

SAMS Backend Architecture: 1. **Layered Architecture**: Presentation → Business Logic → Data Access 2. **Separation of Concerns**: Controllers, Models, Helpers, Middleware 3. **Database Abstraction**: PDO with support for multiple database systems 4. **Security**: Session validation, input validation, encrypted face data 5. **Scalability**: Pagination, indexing, query optimization 6. **Maintainability**: Consistent naming, DRY principles, helper functions

Key Technologies: - PHP 7.4+ with PDO - MySQL / PostgreSQL - Firebase Cloud Messaging - RESTful JSON API - Session-based Authentication